

EA.2 Implementación de Diagrama de clases

Curso:

Programación Orientada a Objetos II

Docente:

Boris Alberto Salleg Royero

Integrantes:

Andrés Felipe Franco Mesa

Jorge Eduardo Zapata Sepúlveda

**Facultad de ingeniería y ciencias agropecuarias
Tecnología en desarrollo de software
INSTITUCIÓN UNIVERSITARIA DIGITAL DE ANTIOQUIA**

Introducción

La Alcaldía de TecnoValle, una ciudad latinoamericana en proceso de transformación digital, movilizó a más de 450.000 pasajeros diarios y genera entre 6 y 12 millones de registros de transporte por día. Su plataforma actual, imperativa, secuencial y altamente acoplada, tarda hasta dos horas en procesar los datos y es propensa a errores de conteo por condiciones de carrera. Por eso nace el proyecto TecnoMóvil Data, que busca evolucionar hacia una arquitectura modular, declarativa y escalable basada en programación funcional.

Este informe documenta el módulo funcional desarrollado como respuesta al caso de estudio de la Unidad 2. El módulo procesa registros de entradas y salidas de estaciones mediante Streams, lambdas, funciones puras y colecciones inmutables, y puede ejecutarse de forma secuencial o paralela sin cambiar los resultados. El documento presenta los objetivos, el desarrollo (diseño, código fuente y explicación de los conceptos aplicados), la evidencia de ejecución y las conclusiones

Objetivos

Objetivo Principal

Diseñar e implementar en Java, sin frameworks externos, un módulo de procesamiento funcional para TecnoMovil Data que analice un conjunto grande de registros simulados de transporte urbano, aplicando Streams, lambdas, funciones puras y colecciones inmutables.

Objetivos específicos.

- Modelar el registro de transporte (idUserio, ruta, estacion, accion, timestamp) como un tipo de datos inmutable.
- Implementar las seis operaciones de análisis del caso (afluencia por estación, horas pico, rutas más utilizadas, patrones de viaje, tiempo promedio entre estaciones y detección de sobrecarga) mediante pipelines declarativos con Streams.
- Aplicar funciones puras, lambdas, referencias a métodos y funciones de orden superior, evitando efectos secundarios y sin modificar las listas originales.
- Permitir que el análisis se ejecute con `stream()` o `parallelStream()` desde un único punto de decisión, garantizando resultados equivalentes.
- Simular un volumen grande de datos (1.000.000 de registros) y comprobar el funcionamiento del módulo mediante una interfaz de consola.
- Documentar el código y sustentar las decisiones de diseño, y extraer conclusiones acordes a los objetivos.

Desarrollo de la actividad

El caso define un registro con cinco campos (idUserio, ruta, estacion, accion con valores “entrada” o “salida”, y timestamp) y seis tareas de análisis: Afluencia por estación (estudio de cuantos usuarios entran a cada estación durante el día), Identificación de horas pico, rutas más utilizadas.

La solución se organizó en cuatro clases con responsabilidades separadas. El núcleo funcional (Registro y AnalizadorTransporte) no imprime ni lee datos; toda la entrada y salida se concentra en Main.

Clases

Main: Es la interfaz de consola con menú. Permite usar 6 datos de prueba o los datos grandes, cambiar entre modo secuencial y paralelo, y mide el tiempo de cada reporte.

Registro: Modelo de datos del caso (record). Inmutable por definición: campos finales, sin setters.

AnalizadorTransporte: Las seis operaciones de análisis. Único estado: el campo final boolean paralelo. El método privado flujo decide entre stream() y parallelStream().

GeneradorDatos: Genera datos simulados de forma determinista. Cada usuario usa new Random(numero) como semilla. $250.000 \text{ usuarios} \times 2 \text{ viajes} \times 2 \text{ registros} = 1.000.000$ de registros, con 6 estaciones y 5 rutas. Las horas de ida (6–9) y regreso (12–21) están ponderadas para producir horas pico realistas.

Código fuente en Java

A continuación se presenta el código completo del proyecto. Solo se usan clases de la biblioteca estándar (java.time, java.util y java.util.stream).

Registro.java

```
import java.time.LocalDateTime;
```

// Al usar 'record', Java hace que todos estos campos sean inmutables (finales) automáticamente.

```
public record Registro(  
    String idUsuario,  
    String ruta,  
    String estacion,  
    String accion, // "entrada" o "salida"  
    LocalDateTime timestamp  
) {}
```

AnalizadorTransporte.java

```
import java.time.Duration;  
import java.util.Collection;  
import java.util.Collections;  
import java.util.Comparator;  
import java.util.List;  
import java.util.Map;  
import java.util.TreeMap;  
import java.util.stream.Collectors;  
import java.util.stream.IntStream;  
import java.util.stream.Stream;  
  
public class AnalizadorTransporte {
```

```
private final boolean paralelo; // final - el analizador también es inmutable
```

```
public AnalizadorTransporte() {
    this(false);
}
```

```
public AnalizadorTransporte(boolean paralelo) {
    this.paralelo = paralelo;
}
```

```
public boolean esParalelo() {
    return paralelo;
}
```

```
// Único punto donde se decide entre stream() y parallelStream()
```

```
private <T> Stream<T> flujo(Collection<T> datos) {
    return paralelo ? datos.parallelStream() : datos.stream();
}
```

```
// a) Afluencia por estación: cuántos usuarios entran a cada estación
```

```
public Map<String, Long> calcularAfluenciaPorEstacion(List<Registro> registros) {
    return flujo(registros)
        .filter(r -> r.accion().equalsIgnoreCase("entrada"))
```

```

        .collect(Collectors.collectingAndThen(
            Collectors.groupingBy(Registro::estacion, TreeMap::new, Collectors.counting()),
            Collections::unmodifiableMap));
    }

```

// b) Horas pico: agrupa por hora y devuelve las horas con más flujo

```

public List<Map.Entry<Integer, Long>> identificarHorasPico(List<Registro> registros, int top)
{
    return flujo(registros)
        .collect(Collectors.groupingBy(r -> r.timestamp().getHour(), Collectors.counting()))
        .entrySet().stream()
        .sorted(Map.Entry.<Integer, Long>comparingByValue().reversed()
            .thenComparing(Map.Entry.comparingByKey())) // desempate: la hora más
temprana
        .limit(top)
        .toList();
}

```

// c) Rutas más utilizadas

```

public List<Map.Entry<String, Long>> rutasMasUtilizadas(List<Registro> registros) {
    return flujo(registros)
        .collect(Collectors.groupingBy(Registro::ruta, Collectors.counting()))
}

```

```

        .entrySet().stream()

        .sorted(Map.Entry.<String, Long>comparingByValue().reversed()

            .thenComparing(Map.Entry.comparingByKey()))

        .toList();
    }

```

// d) Patrones de viaje: por usuario, estaciones en el orden en que aparecen

```

public Map<String, List<String>> patronesViajePorUsuario(List<Registro> registros) {
    return flujo(registros)

        .collect(Collectors.collectingAndThen(

            Collectors.groupingBy(

                Registro::idUsuario,

                TreeMap::new,

                Collectors.mapping(Registro::estacion, Collectors.toUnmodifiableList())),

            Collections::unmodifiableMap));
}

```

// e) Tiempo promedio (en minutos) entre estaciones consecutivas

```

public Map<String, Double> calcularTiempoPromedioEntreEstaciones(List<Registro>
registros) {
    return tramosDeViaje(registros)

        .collect(Collectors.collectingAndThen(

            Collectors.groupingBy(

```

```

        Map.Entry<String, Long>::getKey,
        TreeMap::new,
        Collectors.averagingLong(Map.Entry<String, Long>::getValue)),
        Collections::unmodifiableMap));
    }

    // Promedio general de todos los tramos juntos
    public double calcularTiempoPromedioGeneral(List<Registro> registros) {
        return tramosDeViaje(registros)
            .mapToLong(Map.Entry::getValue)
            .average()
            .orElse(0.0);
    }

    // Convierte los registros en pares (tramo, minutos), uno por cada viaje entrada -> salida
    private Stream<Map.Entry<String, Long>> tramosDeViaje(List<Registro> registros) {
        Map<String, List<Registro>> porUsuario = flujo(registros)
            .collect(Collectors.groupingBy(Registro::idUsuario));

        return flujo(porUsuario.values())
            .flatMap(this::tramosDeUnUsuario);
    }

    private Stream<Map.Entry<String, Long>> tramosDeUnUsuario(List<Registro> delUsuario) {

```

```

// sorted() crea una lista nueva: la original no se toca

List<Registro> ordenados = delUsuario.stream()

    .sorted(Comparator.comparing(Registro::timestamp))

    .toList();

// Se comparan los registros vecinos: (0,1), (1,2), (2,3)...

return IntStream.range(0, ordenados.size() - 1)

    .mapToObj(i -> Map.entry(ordenados.get(i), ordenados.get(i + 1)))

    .filter(par -> esViaje(par.getKey(), par.getValue()))

    .map(par -> Map.entry(

        par.getKey().estacion() + " -> " + par.getValue().estacion(),

        Duration.between(par.getKey().timestamp(),
par.getValue().timestamp()).toMinutes()));

    }

// Un viaje es: "entrada" seguida de "salida" en una estación diferente

private boolean esViaje(Registro inicio, Registro fin) {

    return inicio.accion().equalsIgnoreCase("entrada")

        && fin.accion().equalsIgnoreCase("salida")

        && !inicio.estacion().equals(fin.estacion());

}

// f) Sobrecarga: rutas cuya ocupación (entradas) supera el umbral "críticas"

```

```

public Map<String, Long> detectarSobrecargaEnRutas(List<Registro> registros, long
umbral) {

    return flujo(registros)

        .filter(r -> r.accion().equalsIgnoreCase("entrada"))

        .collect(Collectors.groupingBy(Registro::ruta, Collectors.counting()))

        .entrySet().stream()

        .filter(entrada -> entrada.getValue() > umbral)

        .collect(Collectors.collectingAndThen(

            Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue, Long::sum,
TreeMap::new),

            Collections::unmodifiableMap));

}
}

```

GeneradorDatos.java

```

import java.time.LocalDate;

import java.time.LocalDateTime;

import java.util.List;

import java.util.Random;

import java.util.stream.IntStream;

import java.util.stream.Stream;

// se crea para mantener el código de generación de datos aleatorios separado del resto del
programa

public class GeneradorDatos {

```

```

private static final List<String> ESTACIONES = List.of(
    "Estacion Norte", "Estacion Centro", "Estacion Sur",
    "Estacion Este", "Estacion Oeste", "Estacion Universidad");

private static final List<String> RUTAS = List.of(
    "Ruta A", "Ruta B", "Ruta C", "Ruta D", "Ruta E");

// Las horas repetidas pesan más al sortear para crear horas pico realistas
private static final List<Integer> HORAS_IDA    = List.of(6, 6, 7, 7, 7, 7, 8, 8, 9);
private static final List<Integer> HORAS_REGRESO = List.of(12, 13, 17, 17, 18, 18, 18, 19,
21);

public static List<Registro> generar(int cantidadUsuarios) {
    LocalDate dia = LocalDate.of(2026, 9, 27);

    return IntStream.rangeClosed(1, cantidadUsuarios)
        .boxed()
        .flatMap(numero -> viajesDeUsuario(numero, dia))
        .toList(); // lista inmutable
}

private static Stream<Registro> viajesDeUsuario(int numero, LocalDate dia) {
    Random azar = new Random(numero);

    String idUsuario = String.format("U%06d", numero);

```

```

int origen = azar.nextInt(ESTACIONES.size());

// Suma entre 1 y (n-1) para garantizar que el destino siempre sea distinto al origen
int destino = (origen + 1 + azar.nextInt(ESTACIONES.size() - 1)) % ESTACIONES.size();

String ruta = RUTAS.get(azar.nextInt(RUTAS.size()));

return Stream.concat(
    viaje(idUsuario, ruta, ESTACIONES.get(origen), ESTACIONES.get(destino),
HORAS_IDA, dia, azar),
    viaje(idUsuario, ruta, ESTACIONES.get(destino), ESTACIONES.get(origen),
HORAS_REGRESO, dia, azar));
}

private static Stream<Registro> viaje(String idUsuario, String ruta, String estacionOrigen,
String estacionDestino, List<Integer> horas,
    LocalDate dia, Random azar) {

    LocalDateTime entrada = dia.atTime(horas.get(azar.nextInt(horas.size()))),
azar.nextInt(60));

    LocalDateTime salida = entrada.plusMinutes(10 + azar.nextInt(50)); // viajes de 10 a 59
min

return Stream.of(
    new Registro(idUsuario, ruta, estacionOrigen, "entrada", entrada),
    new Registro(idUsuario, ruta, estacionDestino, "salida", salida));
}

```

```
}
```

Main.java

```
import java.time.LocalDateTime;

import java.util.List;

import java.util.Map;

import java.util.Scanner;

public class Main {

    // Estado de la interfaz (los datos y el modo que el usuario elige en el menú)
    private static List<Registro> registros = datosDePrueba();

    private static AnalizadorTransporte analizador = new AnalizadorTransporte();

    private static long umbralSugerido = 2;

    public static void main(String[] args) {

        try (Scanner teclado = new Scanner(System.in)) {

            ejecutarMenu(teclado);

        }

    }

    private static void ejecutarMenu(Scanner teclado) {

        boolean continuar = true;
```

```

while (continuar) {

    mostrarMenu();

    if (!teclado.hasNextLine()) {

        break; // la entrada se cerró (Ctrl+D / Ctrl+Z): se termina sin error

    }

    String opcion = teclado.nextLine().trim();

    switch (opcion) {

        case "1" -> ejecutar("1) Afluencia por estación (entradas)", () ->

            analizador.calcularAfluenciaPorEstacion(registros).forEach((estacion, cantidad) ->

                System.out.println(" - " + estacion + ": " + cantidad)));

        case "2" -> ejecutar("2) Horas pico (las 3 horas con más movimientos)", () ->

            analizador.identificarHorasPico(registros, 3).forEach(hora ->

                System.out.printf(" - %02d:00 hrs -> %d registros%n", hora.getKey(),
hora.getValue())));

        case "3" -> ejecutar("3) Rutas más utilizadas", () ->

            analizador.rutasMasUtilizadas(registros).forEach(ruta ->

                System.out.println(" - " + ruta.getKey() + ": " + ruta.getValue() + "
movimientos")));

        case "4" -> ejecutar("4) Patrones de viaje por usuario (primeros 10)", () -> {

```

```

        Map<String, List<String>> patrones =
analizador.patronesViajePorUsuario(registros);

        patrones.entrySet().stream().limit(10).forEach(usuario ->

            System.out.println(" - " + usuario.getKey() + ": " + String.join(" | ",
usuario.getValue())));

            System.out.println(" (Usuarios analizados en total: " + patrones.size() + ")");

        });

        case "5" -> ejecutar("5) Tiempo promedio entre estaciones", () -> {

            analizador.calcularTiempoPromedioEntreEstaciones(registros).forEach((tramo,
minutos) ->

                System.out.printf(" - %s: %.1f min%n", tramo, minutos));

            System.out.printf(" >> Promedio general: %.1f min%n",

                analizador.calcularTiempoPromedioGeneral(registros));

        });

        case "6" -> {

            long umbral = leerUmbral(teclado);

            ejecutar("6) Rutas críticas (más de " + umbral + " entradas)", () -> {

                Map<String, Long> criticas = analizador.detectarSobrecargaEnRutas(registros,
umbral);

                if (criticas.isEmpty()) {

                    System.out.println(" - Ninguna ruta reporta sobrecarga crítica.");

                } else {

                    criticas.forEach((ruta, entradas) ->

```

```

        System.out.println(" - ¡ALERTA! " + ruta + " -> " + entradas + " entradas"));
    }
    });
}

case "7" -> {
    registros = datosDePrueba();
    umbralSugerido = 2;
    System.out.println("Datos de prueba cargados: " + registros.size() + " registros.");
}

case "8" -> {
    long inicio = System.currentTimeMillis();
    registros = GeneradorDatos.generar(250_000);
    umbralSugerido = 100_000;
    System.out.println("Datos simulados generados: " + registros.size()
        + " registros (" + (System.currentTimeMillis() - inicio) + " ms).");
}

case "9" -> {
    analizador = new AnalizadorTransporte(!analizador.esParalelo());
    System.out.println("Modo de procesamiento: "
        + (analizador.esParalelo() ? "PARALELO" : "SECUENCIAL"));
}

```

```

        case "0" -> {

            System.out.println("Hasta luego.");

            continuar = false;

        }

        default -> System.out.println("Opción no válida. Elija un número del menú.");

    }

}

// Ejecuta un reporte y muestra cuánto tardó (sirve para comparar secuencial vs paralelo)
private static void ejecutar(String titulo, Runnable reporte) {

    System.out.println("\n--- " + titulo + " ---");

    long inicio = System.nanoTime();

    reporte.run();

    long milisegundos = (System.nanoTime() - inicio) / 1_000_000;

    System.out.println("(" + (analizador.esParalelo() ? "paralelo" : "secuencial")

        + ", " + milisegundos + " ms");

}

private static long leerUmbral(Scanner teclado) {

    System.out.print("Umbral de ocupación [Umbral sugerido " + umbralSugerido + "]: ");

    if (!teclado.hasNextLine()) {

```

```

        return umbralSugerido;
    }

    String texto = teclado.nextLine().trim();

    try {
        return texto.isEmpty() ? umbralSugerido : Long.parseLong(texto);
    } catch (NumberFormatException e) {
        System.out.println("Valor no válido, se usa " + umbralSugerido + ".");
        return umbralSugerido;
    }
}

private static void mostrarMenu() {
    System.out.println("\n----- TECNOMOVIL DATA -----");
    System.out.println("Datos: " + registros.size() + " registros | Modo: "
        + (analizador.esParalelo() ? "PARALELO" : "SECUENCIAL"));
    System.out.println("1. Afluencia por estación");
    System.out.println("2. Horas pico");
    System.out.println("3. Rutas más utilizadas");
    System.out.println("4. Patrones de viaje por usuario");
    System.out.println("5. Tiempo promedio entre estaciones");
    System.out.println("6. Detección de sobrecarga en rutas");
    System.out.println("7. Usar datos de prueba (6 registros)");
    System.out.println("8. Generar datos simulados grandes (1.000.000 registros)");
    System.out.println("9. Cambiar modo secuencial / paralelo");
}

```

```

        System.out.println("0. Salir");

        System.out.print("Opción: ");

    }

    // Datos simulados (quemados) para prueba
    private static List<Registro> datosDePrueba() {

        return List.of(

            new Registro("U001", "Ruta A", "Estacion Norte", "entrada", LocalDateTime.of(2026, 9,
27, 7, 0)),

            new Registro("U001", "Ruta A", "Estacion Centro", "salida", LocalDateTime.of(2026, 9,
27, 7, 30)), // 30 mins

            new Registro("U002", "Ruta B", "Estacion Sur", "entrada", LocalDateTime.of(2026, 9,
27, 8, 15)),

            new Registro("U002", "Ruta B", "Estacion Centro", "salida", LocalDateTime.of(2026, 9,
27, 8, 45)), // 30 mins

            new Registro("U003", "Ruta A", "Estacion Norte", "entrada", LocalDateTime.of(2026, 9,
27, 7, 10)),

            new Registro("U004", "Ruta A", "Estacion Norte", "entrada", LocalDateTime.of(2026, 9,
27, 7, 15))

        );

    }
}

```

Documentación de los conceptos de programación funciones

Funciones puras

Una función es **pura** cuando su resultado depende únicamente de sus argumentos y no produce efectos secundarios: no modifica variables externas, no imprime, no lee del teclado y no altera los datos recibidos. Con los mismos argumentos siempre devuelve el mismo resultado, lo que la hace segura para ejecutarse en paralelo.

Función	Por qué es pura
esViaje(inicio, fin)	Solo lee sus dos parámetros (inmutables) y devuelve un boolean. No depende de variables globales, reloj ni consola.
tramosDeUnUsuario(delUsuario)	Ordena una copia de la lista (sorted()) no altera la original) y devuelve un nuevo Stream de pares (tramo, minutos). La misma entrada produce la misma salida.
calcularAfluenciaPorEstacion, identificarHorasPico, rutasMasUtilizadas, patronesViajePorUsuario, calcularTiempoPromedio..., detectarSobrecargaEnRutas	Cada una construye un pipeline nuevo a partir de la lista recibida y devuelve una colección nueva e inmutable. No escriben en campos, no imprimen y no modifican la lista de entrada.
flujo(datos)	Depende solo de su argumento y del campo final paralelo, que nunca cambia: es determinista.
GeneradorDatos.viajesDeUsuario y generar	Son deterministas: el Random se crea localmente con la semilla del usuario, así que los mismos parámetros generan siempre los mismos registros. El método auxiliar viaje avanza el estado de ese Random local, una mutación acotada a la generación de un solo usuario.

Inmutabilidad

Los datos no cambian después de creados: las operaciones producen valores nuevos en lugar de modificar los existentes. Esto evita las condiciones de carrera que afectaban al sistema anterior, porque varios hilos pueden leer los mismos objetos sin sincronización.

Mecanismo	Dónde se usa	Qué garantiza
record Registro	Registro.java	Campos privados y finales, sin setters. String y LocalDateTime también son inmutables.
private final boolean paralelo	AnalizadorTransporte	Cambiar de modo crea un analizador nuevo (new AnalizadorTransporte (!analizador.esParalelo())) en vez de modificar el existente.
List.of(...)	datosDePrueba(), constantes de GeneradorDatos	Listas inmutables: add o remove lanzan UnsupportedOperationException.
Stream.toList()	generar, identificarHorasPico, rutasMasUtilizadas, tramosDeUnUsuario	Devuelve una lista no modificable.
Collectors.collectingAndThen(..., Collections::unmodifiableMap)	Operaciones a, d, e y f	El mapa resultante se entrega envuelto como no modificable.
Collectors.toUnmodifiableList()	patronesViajePorUsuario	Las listas de estaciones de cada usuario son inmutables.
sorted() sobre un stream	tramosDeUnUsuario, identificarHorasPico, rutasMasUtilizadas	Ordena produciendo un resultado nuevo; la lista original conserva su orden.

Funciones de orden superior

Una **función de orden superior** recibe otras funciones como parámetro, devuelve una función, o ambas cosas. Son la base del estilo declarativo: se describe qué transformar y la función recibida define cómo.

Función de orden superior	Dónde se usa	Función que recibe o devuelve
Stream.filter	Operaciones a y f	Recibe un Predicate (por ejemplo, <code>r -> r.accion().equalsIgnoreCase("entrada")</code>).
Stream.map / mapToObj	Operación e, GeneradorDatos	Reciben una Function que transforma cada elemento.
Stream.flatMap	Operación e, GeneradorDatos.generar	Recibe una función que convierte cada elemento en un Stream, y los aplana.
Stream.sorted(Comparator)	Operaciones b, c y e	Recibe un comparador compuesto con <code>comparingByValue().reversed()</code> <code>.thenComparing(...)</code> .
Comparator.comparing (Registro::timestamp)	Operación e	Recibe una función extractora y devuelve un Comparator (devuelve una función).
Collectors.groupingBy (clasificador, fábrica, downstream)	Operaciones a a f	Recibe una función clasificadora, un Supplier de mapa y otro colector.
Collectors.collectingAndThen(colector, finalizador)	Operaciones a, d, e y f	Recibe un colector y una función final (<code>Collections::unmodifiableMap</code>).

Función de orden superior	Dónde se usa	Función que recibe o devuelve
Collectors.mapping y Collectors.toMap	Operaciones d y f	mapping recibe una función y un colector; toMap recibe funciones de clave, valor y combinación (Long::sum).
ejecutar(String titulo, Runnable reporte)	Main (función propia)	Recibe el reporte como lambda, lo ejecuta y mide el tiempo alrededor de él.

Lambdasy referencia de métodos

Una **lambda** es una función anónima escrita como expresión (parámetros -> cuerpo) que implementa una interfaz funcional. Una **referencia a método** (Clase::método) es una forma abreviada cuando la lambda solo invoca un método existente.

Lambda	Ubicación	Interfaz funcional y función que cumple
r -> r.accion().equalsIgnoreCase("entrada")	Operaciones a y f	Predicate<Registro>: conserva solo las entradas.
r -> r.timestamp().getHour()	Operación b	Function<Registro, Integer>: clasifica cada registro por su hora.
entrada -> entrada.getValue() > umbral	Operación f	Predicate<Map.Entry<String, Long>>: conserva las rutas sobre el umbral. Captura la variable umbral , que es efectivamente final.
i -> Map.entry(ordenados.get(i), ordenados.get(i + 1))	Operación e	IntFunction: arma el par de registros vecinos (i, i+1).

Lambda	Ubicación	Interfaz funcional y función que cumple
par -> esViaje(par.getKey(), par.getValue())	Operación e	Predicate: deja solo los pares entrada → salida en estaciones distintas.
par -> Map.entry(... + " -> " + ..., Duration.between(...).toMinutes())	Operación e	Function: convierte el par en (tramo, minutos).
numero -> viajesDeUsuario(numero, dia)	GeneradorDatos.gen erar	Function<Integer, Stream<Registro>>: genera los registros de cada usuario.
() -> analizador.calcular...(registros)...	Main, opciones 1 a 6	Runnable: describe el reporte que ejecutar cronometra.
(estacion, cantidad) -> System.out.println(...)	Main	BiConsumer para Map.forEach: imprime cada par clave-valor.

Referencia a método	Tipo	Uso en el proyecto
Registro::estacion, Registro::ruta, Registro::idUserio, Registro::timestamp	Método de instancia (accesores del record)	Clasificadores de groupingBy, extractor de Comparator.comparing y de mapping.
TreeMap::new	Constructor (Supplier)	Fábrica del mapa ordenado por clave en groupingBy y toMap.
Collections::unmodifiableMap	Método estático	Finalizador de collectingAndThen.
Long::sum	Método estático (BinaryOperator)	Función de combinación de toMap.
this::tramosDeUnUsuario	Método de instancia de un objeto concreto	Función de flatMap en tramosDeViaje.

Referencia a método	Tipo	Uso en el proyecto
<code>Map.Entry::getKey</code> / <code>Map.Entry::getValue</code>	Método de instancia de un tipo	Extracción de clave y valor en <code>toMap</code> , <code>mapToLong</code> y <code>averagingLong</code> .

Streams

Un **Stream** representa una secuencia de elementos sobre la que se define un pipeline: una **fuentes**, cero o más **operaciones intermedias** (`filter`, `map`, `flatMap`, `sorted`, `limit`), que son perezosas, y una **operación terminal** (`collect`, `toList`, `average`) que dispara la ejecución. Un stream no almacena datos ni modifica su fuente.

Operación	Fuente	Operaciones intermedias	Operación terminal
a) Afluencia	flujo (registros)	<code>filter</code> (entradas)	<code>collect(collectingAndThen(groupingBy(..., counting())))</code>
b) Horas pico	flujo(registros)	Tras agrupar: <code>entrySet().stream()</code> , <code>sorted</code> , <code>limit</code>	<code>collect(groupingBy)</code> y <code>toList()</code>
c) Rutas	flujo(registros)	Tras agrupar: <code>entrySet().stream()</code> , <code>sorted</code>	<code>collect(groupingBy)</code> y <code>toList()</code>
d) Patrones	flujo(registros)	Ninguna (todo en el colector)	<code>collect(groupingBy(..., mapping(..., toUnmodifiableList())))</code>
e) Tiempo promedio	flujo(registros) y luego <code>flujo(porUsuario.valores())</code>	<code>flatMap</code> , <code>sorted</code> , <code>IntStream.range</code> , <code>mapToObj</code> , <code>filter</code> , <code>map</code>	<code>collect(groupingBy(..., averagingLong)); average()</code> para el promedio general

Operación	Fuente	Operaciones intermedias	Operación terminal
f) Sobrecarga	flujo(registros)	filter, y luego entrySet().stream().filter	collect(groupingBy) y collect(toMap)

Evidencia de ejecución

Las capturas corresponden a una única sesión de consola: primero con los 6 registros de prueba, luego con 1.000.000 de registros simulados en modo secuencial y finalmente en modo paralelo (opción 9). Para ahorrar espacio, en las figuras 2 a 9 se abrevia la lista de opciones del menú, que se repite en cada iteración.

```
Datos: 6 registros | Modo: SECUENCIAL
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 2

--- 2) Horas pico (las 3 horas con más movimientos) ---
- 07:00 hrs -> 4 registros
- 08:00 hrs -> 2 registros
(secuencial, 8 ms)

----- TECNOMOVIL DATA -----
Datos: 6 registros | Modo: SECUENCIAL
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 3

--- 3) Rutas más utilizadas ---
- Ruta A: 4 movimientos
- Ruta B: 2 movimientos
(secuencial, 1 ms)
```

Figura 1. Menú principal y reportes 1 (afluencia por estación), 2 (horas pico) y 3 (rutas más utilizadas) con los datos de prueba.

```

----- TECNOMOVIL DATA -----
Datos: 6 registros | Modo: SECUENCIAL
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 4

--- 4) Patrones de viaje por usuario (primeros 10) ---
- U001: Estacion Norte | Estacion Centro
- U002: Estacion Sur | Estacion Centro
- U003: Estacion Norte
- U004: Estacion Norte
(Usuarios analizados en total: 4)
(secuencial, 6 ms)

----- TECNOMOVIL DATA -----
Datos: 6 registros | Modo: SECUENCIAL
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 5

--- 5) Tiempo promedio entre estaciones ---
- Estacion Norte -> Estacion Centro: 30,0 min
- Estacion Sur -> Estacion Centro: 30,0 min
>> Promedio general: 30,0 min
(secuencial, 13 ms)

----- TECNOMOVIL DATA -----
Datos: 6 registros | Modo: SECUENCIAL
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 6
Umbral de ocupación [Umbral sugerido 2]: 2

--- 6) Rutas críticas (más de 2 entradas) ---
- ¡ALERTA! Ruta A -> 3 entradas
(secuencial, 3 ms)

```

Figura 2. Reportes 4 (patrones de viaje), 5 (tiempo promedio) y 6 (rutas críticas) con los datos de prueba. En el reporte 6 se pulsó Enter para aceptar el umbral sugerido (2).

U001 y U002 registran entrada y salida; sus tramos duran 30 minutos, de ahí el promedio general de 30,0 min. U003 y U004 solo tienen una entrada, por lo que no forman un

viaje completo y el cálculo los ignora. Con umbral 2, la Ruta A (3 entradas) se marca como crítica.

```

----- TECNOMOVIL DATA -----
Datos: 6 registros | Modo: SECUENCIAL
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 8
Datos simulados generados: 1000000 registros (219 ms).

----- TECNOMOVIL DATA -----
Datos: 1000000 registros | Modo: SECUENCIAL
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 1

--- 1) Afluencia por estación (entradas) ---
- Estacion Centro: 83333
- Estacion Este: 83346
- Estacion Norte: 83322
- Estacion Oeste: 83338
- Estacion Sur: 83327
- Estacion Universidad: 83334
(secuencial, 35 ms)

----- TECNOMOVIL DATA -----
Datos: 1000000 registros | Modo: SECUENCIAL
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 2

--- 2) Horas pico (las 3 horas con más movimientos) ---
- 07:00 hrs -> 190334 registros
- 18:00 hrs -> 150742 registros
- 08:00 hrs -> 143033 registros
(secuencial, 27 ms)

```

Figura 3. Opción 8 (generación de 1.000.000 de registros) y reportes 1 y 2 en modo secuencial.

El generador produjo 1.000.000 de registros en 1.273 ms. Cada estación recibe cerca de 83.300 entradas, una distribución uniforme. Las horas pico son las 07:00 (190.334 movimientos), las 18:00 (150.742) y las 08:00 (143.033), coherentes con las horas ponderadas del generador.

```

----- TECNOMOVIL DATA -----
Datos: 1000000 registros | Modo: SECUENCIAL
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 3

--- 3) Rutas más utilizadas ---
- Ruta A: 200036 movimientos
- Ruta C: 200024 movimientos
- Ruta D: 199988 movimientos
- Ruta E: 199980 movimientos
- Ruta B: 199972 movimientos
(secuencial, 40 ms)

----- TECNOMOVIL DATA -----
Datos: 1000000 registros | Modo: SECUENCIAL
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 4

--- 4) Patrones de viaje por usuario (primeros 10) ---
- U000001: Estacion Este | Estacion Centro | Estacion Centro | Estacion Este
- U000002: Estacion Oeste | Estacion Centro | Estacion Centro | Estacion Oeste
- U000003: Estacion Sur | Estacion Este | Estacion Este | Estacion Sur
- U000004: Estacion Sur | Estacion Universidad | Estacion Universidad | Estacion Sur
- U000005: Estacion Universidad | Estacion Sur | Estacion Sur | Estacion Universidad
- U000006: Estacion Centro | Estacion Este | Estacion Este | Estacion Centro
- U000007: Estacion Oeste | Estacion Este | Estacion Este | Estacion Oeste
- U000008: Estacion Oeste | Estacion Norte | Estacion Norte | Estacion Oeste
- U000009: Estacion Centro | Estacion Este | Estacion Este | Estacion Centro
- U000010: Estacion Este | Estacion Oeste | Estacion Oeste | Estacion Este
(Usuarios analizados en total: 250000)
(secuencial, 187 ms)

```

Figura 4. Reportes 3 (rutas más utilizadas) y 4 (patrones de viaje) en modo secuencial.

Las cinco rutas tienen cerca de 200.000 movimientos cada una (la Ruta A lidera con 200.036). En el reporte 4 se analizaron 250.000 usuarios y cada patrón muestra origen, destino, destino y origen, lo que refleja el viaje de ida y el de regreso.

```

----- TECNOMOVIL DATA -----
Datos: 1000000 registros | Modo: SECUENCIAL
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 5

--- 5) Tiempo promedio entre estaciones ---
- Estacion Centro -> Estacion Este: 34,5 min
- Estacion Centro -> Estacion Norte: 34,6 min
- Estacion Centro -> Estacion Oeste: 34,5 min
- Estacion Centro -> Estacion Sur: 34,6 min
- Estacion Centro -> Estacion Universidad: 34,5 min
- Estacion Este -> Estacion Centro: 34,4 min
- Estacion Este -> Estacion Norte: 34,5 min
- Estacion Este -> Estacion Oeste: 34,5 min
- Estacion Este -> Estacion Sur: 34,6 min
- Estacion Este -> Estacion Universidad: 34,5 min
- Estacion Norte -> Estacion Centro: 34,4 min
- Estacion Norte -> Estacion Este: 34,6 min
- Estacion Norte -> Estacion Oeste: 34,5 min
- Estacion Norte -> Estacion Sur: 34,4 min
- Estacion Norte -> Estacion Universidad: 34,5 min
- Estacion Oeste -> Estacion Centro: 34,6 min
- Estacion Oeste -> Estacion Este: 34,5 min
- Estacion Oeste -> Estacion Norte: 34,4 min
- Estacion Oeste -> Estacion Sur: 34,5 min
- Estacion Oeste -> Estacion Universidad: 34,6 min
- Estacion Sur -> Estacion Centro: 34,4 min
- Estacion Sur -> Estacion Este: 34,5 min
- Estacion Sur -> Estacion Norte: 34,6 min
- Estacion Sur -> Estacion Oeste: 34,5 min
- Estacion Sur -> Estacion Universidad: 34,5 min
- Estacion Universidad -> Estacion Centro: 34,6 min
- Estacion Universidad -> Estacion Este: 34,6 min
- Estacion Universidad -> Estacion Norte: 34,5 min
- Estacion Universidad -> Estacion Oeste: 34,5 min
- Estacion Universidad -> Estacion Sur: 34,4 min
>> Promedio general: 34,5 min
(secuencial, 364 ms)

```

Figura 5. *Reporte 5 (tiempo promedio entre estaciones) en modo secuencial.*

Se calcularon los 30 tramos posibles entre las 6 estaciones, todos cercanos a 34,5 minutos; el promedio general también es 34,5 min. Este valor coincide con el esperado, ya que el generador asigna duraciones uniformes entre 10 y 59 minutos.

Cambio a modo paralelo

```

----- TECNOMOVIL DATA -----
Datos: 1000000 registros | Modo: SECUENCIAL
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 6
Umbral de ocupación [Umbral sugerido 100000]: 100000

--- 6) Rutas críticas (más de 100000 entradas) ---
- ¡ALERTA! Ruta A -> 100018 entradas
- ¡ALERTA! Ruta C -> 100012 entradas
(secuencial, 19 ms)

----- TECNOMOVIL DATA -----
Datos: 1000000 registros | Modo: SECUENCIAL
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 9
Modo de procesamiento: PARALELO

----- TECNOMOVIL DATA -----
Datos: 1000000 registros | Modo: PARALELO
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 1

--- 1) Afluencia por estación (entradas) ---
- Estacion Centro: 83333
- Estacion Este: 83346
- Estacion Norte: 83322
- Estacion Oeste: 83338
- Estacion Sur: 83327
- Estacion Universidad: 83334
(paralelo, 32 ms)

```

Figura 6. Reporte 6 en modo secuencial (umbral 100.000), cambio a modo PARALELO (opción 9) y reportes 1 en paralelo.

Con el umbral sugerido de 100.000 entradas, las rutas A (100.018) y C (100.012) se marcan como críticas. Tras la opción 9, el encabezado del menú pasa a indicar “Modo: PARALELO”.

```

----- TECNOMOVIL DATA -----
Datos: 1000000 registros | Modo: PARALELO
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 3

--- 3) Rutas más utilizadas ---
- Ruta A: 200036 movimientos
- Ruta C: 200024 movimientos
- Ruta D: 199988 movimientos
- Ruta E: 199980 movimientos
- Ruta B: 199972 movimientos
(paralelo, 13 ms)

----- TECNOMOVIL DATA -----
Datos: 1000000 registros | Modo: PARALELO
1. Afluencia por estación
2. Horas pico
3. Rutas más utilizadas
4. Patrones de viaje por usuario
5. Tiempo promedio entre estaciones
6. Detección de sobrecarga en rutas
7. Usar datos de prueba (6 registros)
8. Generar datos simulados grandes (1.000.000 registros)
9. Cambiar modo secuencial / paralelo
0. Salir
Opción: 4

--- 4) Patrones de viaje por usuario (primeros 10) ---
- U000001: Estacion Este | Estacion Centro | Estacion Centro | Estacion Este
- U000002: Estacion Oeste | Estacion Centro | Estacion Centro | Estacion Oeste
- U000003: Estacion Sur | Estacion Este | Estacion Este | Estacion Sur
- U000004: Estacion Sur | Estacion Universidad | Estacion Universidad | Estacion Sur
- U000005: Estacion Universidad | Estacion Sur | Estacion Sur | Estacion Universidad
- U000006: Estacion Centro | Estacion Este | Estacion Este | Estacion Centro
- U000007: Estacion Oeste | Estacion Este | Estacion Este | Estacion Oeste
- U000008: Estacion Oeste | Estacion Norte | Estacion Norte | Estacion Oeste
- U000009: Estacion Centro | Estacion Este | Estacion Este | Estacion Centro
- U000010: Estacion Este | Estacion Oeste | Estacion Oeste | Estacion Este
(Usuarios analizados en total: 250000)
(paralelo, 113 ms)

```

Figura 7. Reportes 3 y 4 en modo paralelo.

Resultado de la verificación

Los seis reportes con 1.000.000 de registros producen exactamente los mismos resultados en modo secuencial y en modo paralelo. Esto confirma que las funciones puras, las colecciones inmutables y los colectores permiten paralelizar sin condiciones de carrera ni errores de conteo.

Tiempos observados

los tiempos corresponden a una sola ejecución y dependen del equipo, de la cantidad de núcleos y del calentamiento de la JVM (compilación JIT). En esta sesión solo había 1 núcleo disponible y

los reportes secuenciales se ejecutaron primero, con la JVM aún “fría”, por lo que la diferencia no debe atribuirse únicamente al paralelismo. La evidencia sólida es la equivalencia de resultados; para medir la aceleración real conviene repetir cada reporte varias veces en un equipo con varios núcleos.

Conclusiones

Se modeló el registro como tipo inmutable, el record Registro entrega inmutabilidad con una sola declaración, elimina setters y hace seguro compartir los registros entre hilos.

Se implementó las seis operaciones con pipelines operativos. Esto hizo que cada operación se expresa como un pipeline corto (filtrar, agrupar, contar u ordenar) sin bucles ni acumuladores mutables, y resulta más legible que un equivalente imperativo.

Se aplicaron funciones puras, lambdas, referencias a métodos y funciones de orden superior. Las funciones auxiliares (esViaje, tramosDeUnUsuario) son puras y las lambdas y referencias a métodos describen el comportamiento de cada paso. La función propia ejecutar mostró cómo pasar comportamiento como parámetro para reutilizar la medición de tiempo.

Se permite ejecución secuencial o paralela. Con un único punto de decisión (flujo) el mismo código corre con stream() o parallelStream(), y los seis reportes dieron resultados idénticos en ambos modos.

Se simuló un volumen grande y comprobar el funcionamiento. GeneradorDatos produjo 1.000.000 de registros reproducibles y el módulo los procesó en tiempos de entre 24 ms y 1,1 s por reporte en esta sesión.

Se documenta y sustentan las decisiones. Separar el núcleo funcional de la interfaz de consola mantuvo los efectos secundarios en un solo lugar y facilita integrar el módulo en TecnoMovil Data.

Anexos

Link de repositorio público:

<https://github.com/Joziudigital/EA1.-Diagramas-de-clases/tree/main/EA3-Programaci%C3%B3n%20Funcional>

Link de archivo .zip:

<https://drive.google.com/file/d/1mdY27wGO6JaOW3BAY1Y-FP6dzhNg9XJ3/view?usp=sharing>

Link de documento:

Referencias

- Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.
- Meyer, B. (1997). *Object-oriented software construction* (2.^a ed.). Prentice Hall.
- Oracle. (s. f.). *Object-oriented programming concepts*. The Java Tutorials.
<https://docs.oracle.com/javase/tutorial/java/concepts/>

Enlaces